

La IA que Diseña IA: Una Perspectiva de Optimización Multiobjetivo en NAS.

DR. COSIJOPII GARCÍA GARCÍA

Ingeniería en desarrollo de software y sistemas inteligentes

Universidad del Istmo Campus Ixtepec

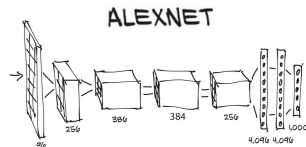
21 de Septiembre 2026



Motivación

¿Por qué automatizar el diseño de redes neuronales?

- ▶ Históricamente, el Machine Learning tradicional requería **Feature Engineering** (diseño manual de características).
- ▶ El Deep Learning eliminó eso, pero introdujo un nuevo cuello de botella: el **Architecture Engineering** (diseño manual de topologías).
- ▶ Los grandes avances en visión y lenguaje (AlexNet, ResNet, **Transformers**) son producto de años de intuición, prueba y error por parte de expertos.



Fuente: Elsken, T., Metzen, J. H., & Hutter, F. (2019). "Neural Architecture Search: A Survey". *Journal of Machine Learning Research (JMLR)*, Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). "ImageNet classification with deep convolutional neural networks". *NeurIPS*.

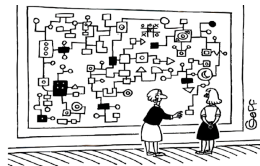
El problema de diseñar arquitecturas a mano

- ▶ **Espacio de búsqueda intratable:** Elegir el número de capas, tipos de convolución, tamaños de filtro y conexiones genera un espacio de búsqueda gigante.
- ▶ **Sesgo cognitivo:** Los humanos tienden a diseñar redes simétricas, modulares y fáciles de comprender, ignorando topologías asimétricas que podrían ser más eficientes.
- ▶ **Costo computacional:** Entrenar y probar una sola arquitectura desde cero puede tomar días en múltiples GPUs.

Fuente: Zoph, B., & Le, Q. V. (2017). "Neural Architecture Search with Reinforcement Learning". *ICLR*.

De programar la red a programar la búsqueda

- ▶ **El cambio de paradigma:** En lugar de usar nuestra inteligencia para armar la mejor red, diseñamos algoritmos que lo hagan por nosotros. ← **Esta idea no es nueva.**
- ▶ **¿Qué es NAS?** *Neural Architecture Search* es la disciplina dedicada a automatizar el diseño de redes neuronales.
- ▶ **Beneficios prácticos:** Modelos con mayor precisión, menor tamaño y adaptados a problemas específicos sin requerir de un ingeniero artesano.



Fuente: Miller, G. F., Todd, P. M., & Hegde, S. U. (1989). "Designing Neural Networks using Genetic Algorithms". *ICGA*,
LeCun, Y., Denker, J., & Solla, S. (1990). "Optimal brain damage". *Advances in neural information processing systems (NeurIPS)*, 2.,
Stanley, K. O., & Miikkulainen, R. (2002). "Evolving neural networks through augmenting topologies". *Evolutionary computation*.

Cómo funciona NAS

Los tres componentes de un sistema NAS

Todo algoritmo NAS se define por tres pilares fundamentales que interactúan en un ciclo cerrado:

1. **Espacio de Búsqueda (Search Space):** Define qué arquitecturas pueden construirse.
2. **Estrategia de Búsqueda (Search Strategy):** El algoritmo que decide cómo explorar ese espacio de manera eficiente.
3. **Estimación de Rendimiento (Performance Estimation):** El método para evaluar qué tan buena es una arquitectura generada.

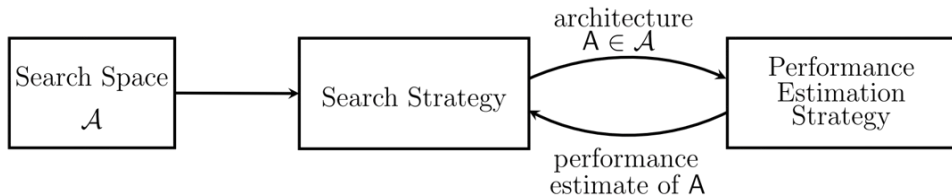
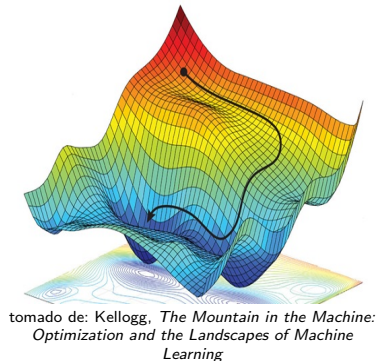


Figura: Los tres componentes de un sistema NAS

Fuente: Elsken, Metzen y Hutter, Neural Architecture Search: A Surve

1. El espacio de búsqueda

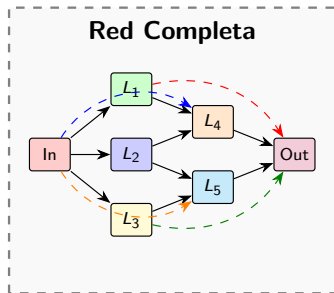
- ▶ **Macro-arquitectura:** Búsqueda capa por capa de toda la red. Genera un espacio inmenso y muy costoso de explorar.
- ▶ **Micro-arquitectura (Basado en Celdas):** Se busca únicamente la estructura de una celda pequeña (Normal y de Reducción). Luego, esta celda se apila en serie para formar la red completa.
- ▶ **Ventaja:** Reduce drásticamente la dificultad de la búsqueda y permite escalar la red final según el hardware.



Fuente: Zoph, B., et al. (2018). "Learning Transferable Architectures for Scalable Image Recognition". *CVPR*.

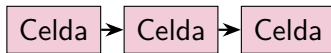
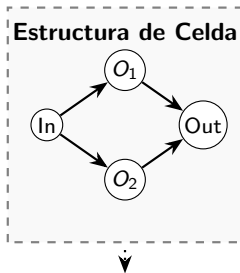
Visualizando los Espacios de Búsqueda

Macro-arquitectura



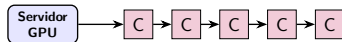
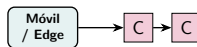
Búsqueda global.
Espacio inmenso y
sin estructura fija.

Micro-arquitectura



Se busca una
celda y se apila
repetidamente.

Ventaja



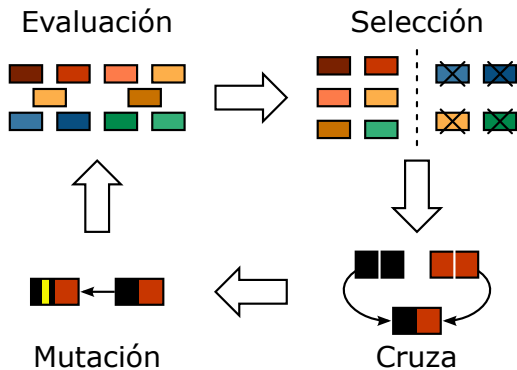
Escalar el
tamaño de la
red depende del
hardware objetivo.

2. La estrategia de búsqueda

- ▶ **Aprendizaje por Refuerzo (RL):** Un controlador (como una RNN) escribe la arquitectura y usa la precisión obtenida como recompensa.
- ▶ **Computación Evolutiva (EA):** Aplica mutación y cruza sobre una población de arquitecturas inspirándose en la biología.
- ▶ **Basada en Gradiente (DARTS):** Relaja el espacio discreto convirtiéndolo en continuo, permitiendo que la arquitectura se aprenda usando *Backpropagation*.

Fuente: Liu, H., Simonyan, K., & Yang, Y. (2019). "DARTS: Differentiable Architecture Search". *ICLR*.

Computación Evolutiva (EA)



(Ciclo evolutivo)

3. Cómo evaluar los modelos rápidamente

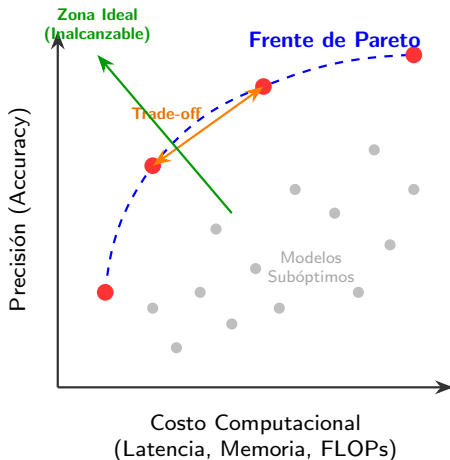
- ▶ Entrenar cada arquitectura generada al 100 % es inviable computacionalmente (**Aunque si se puede**).
- ▶ **Soluciones modernas:**
 - ▶ **Compartición de pesos (Weight Sharing):** Entrenar una “Super-red” y heredar pesos a las sub-redes.
 - ▶ **Entrenamiento parcial:** Entrenar con menos épocas o bases de datos reducidas.
 - ▶ **Zero-cost proxies:** Estimar el rendimiento del modelo analizando sus gradientes en los primeros segundos, sin necesidad de entrenarlo.

Fuente: Mellor, J., et al. (2021). “Neural Architecture Search without Training”. *ICML*.

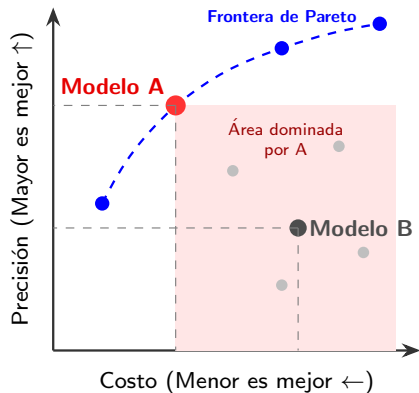
Optimización Multiobjetivo

La precisión no lo es todo: El reto multiobjetivo

- ▶ En aplicaciones del mundo real (dispositivos móviles, IoT, robótica), una precisión del 99 % no sirve si la inferencia es lenta o drena la batería.
- ▶ **Objetivos en conflicto (Trade-offs):**
 - ▶ Exactitud / Precisión.
 - ▶ Memoria (Cantidad de parámetros).
 - ▶ Consumo de energía (FLOPs).
 - ▶ Latencia (Tiempo de respuesta).



Frentes de Pareto y soluciones óptimas

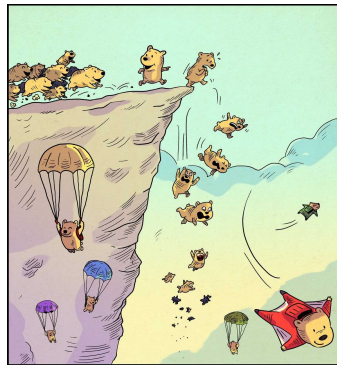


- ▶ ¿Cómo elegimos al “mejor” modelo si hay múltiples objetivos?
- ▶ **Dominancia de Pareto:** Una solución domina a otra si es estrictamente mejor en al menos un objetivo, y no es peor en el resto.
- ▶ **Frontera de Pareto:** El conjunto de todas las soluciones *no dominadas*. Representa el límite físico del compromiso óptimo entre precisión y eficiencia.

Fuente: Deb, K., et al. (2002). “A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II”. *IEEE Transactions on Evolutionary Computation*.

¿Qué significa que un algoritmo sea “Evolutivo”?

- ▶ Se basan en el principio de la **supervivencia del más apto** (Charles Darwin).
- ▶ Las redes mejor evaluadas se seleccionan para “reproducirse” mediante **Cruza** y **Mutación**.
- ▶ Este proceso se repite hasta cumplir un criterio de paro determinado.



Tomado de: Ha, “A Visual Guide to Evolution Strategies”

Algoritmos Evolutivos (MOEAs) aplicados a NAS

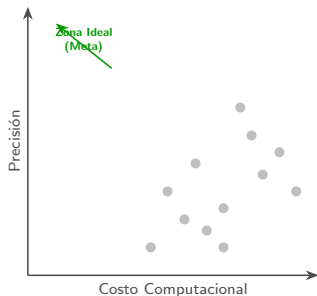
- ▶ Los MOEAs son ideales para NAS porque no requieren derivadas y manejan la naturaleza discreta de los grafos sin problemas.
- ▶ **Flujo clásico en NAS Multiobjetivo:**
 1. Generar población inicial de redes aleatorias.
 2. Evaluarlas en precisión y costo computacional (FLOPs).
 3. Seleccionar las mejores, utilizando por ejemplo (*Non-dominated sorting*).
 4. Aplicar cruza y mutación para evolucionar la población.

Fuente: Lu, Z., et al. (2019). "NSGA-Net: Neural Architecture Search using Multi-Objective Genetic Algorithm". *GECCO*.

Cómo evoluciona una población

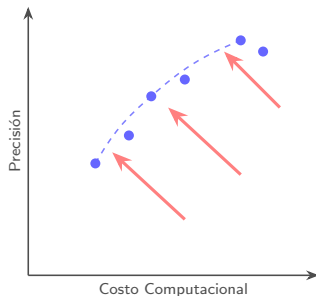
1. Inicialización

Modelos dispersos y costosos



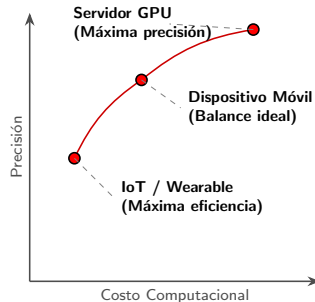
2. Convergencia

Empujando la Frontera de Pareto



3. Soluciones Óptimas

Elección según el Hardware



Aplicaciones y Futuro

NAS en el mundo real: Casos de éxito

- ▶ **Smartphones (Fotografía y Visión):** Modelos icónicos como *MobileNetV3* fueron descubiertos usando NAS. Permiten reconocimiento facial y filtros en tiempo real sin agotar la batería del teléfono.
- ▶ **Salud y Wearables (Edge AI):** Relojes inteligentes que analizan datos en tiempo real. NAS encuentra redes diminutas que caben en unos pocos kilobytes de memoria RAM (Microcontroladores).
- ▶ **Conducción Autónoma:** Detección de peatones y carriles en milisegundos. NAS optimiza arquitecturas para ejecutarse en hardware específico (ej. chips de NVIDIA), donde un retraso en latencia es fatal.



Fuentes clave: - Howard, A., et al. (2019). "Searching for MobileNetV3". *ICCV*. (Smartphones)
- Lin, J., et al. (2020). "MCUNet: Tiny Deep Learning on IoT Devices". *NeurIPS*. (Edge AI)
- Cai, H., et al. (2018). "ProxylessNAS: Direct Neural Architecture Search on Target Task and Hardware". *ICLR*. (Hardware específico)

La frontera actual: NAS en Transformers y LLMs

- ▶ Habiendo conquistado la visión artificial (CNNs), el paradigma NAS se expande ahora a los modelos de lenguaje masivos.
- ▶ **Auto-Transformers:** Búsqueda automatizada del número óptimo de cabezas de atención, dimensiones ocultas y topología de capas para maximizar el rendimiento en hardware específico.
- ▶ **Modelos de Lenguaje (LLMs):** NAS se aplica hoy para encontrar sub-redes altamente eficientes (*Pruning, Mixture of Experts*) que reduzcan la masiva huella de memoria en servidores sin perder capacidad de razonamiento.

Fuentes clave:

- So, D., et al. (2019). "The Evolved Transformer". *ICML*. (Búsqueda de arquitectura Transformer)
- Wang, H., et al. (2020). "HAT: Hardware-Aware Transformers for Efficient Natural Language Processing". *ACL*. (Transformers eficientes)

Aplicación en hardware limitado (Edge AI)

- ▶ **Hardware-Aware NAS:** La búsqueda incluye las especificaciones físicas del dispositivo (latencia real en un microcontrolador, Raspberry Pi o FPGA) como restricción fuerte.
- ▶ **Once-for-All (OFA):** Entrenar una “Super-red” gigante de una sola vez, y usar NAS multiobjetivo para extraer rápidamente sub-redes adaptadas a cada dispositivo sin tener que re-entrenarlas.

Fuente: Cai, H., et al. (2019). “Once-for-All: Train One Network and Specialize it for Efficient Deployment”. *ICLR*.

Retos actuales y qué falta por resolver

- ▶ **Sustentabilidad (Green AI):** Las primeras búsquedas NAS costaban cientos de miles de dólares en energía. Reducir la huella de carbono de la búsqueda es un área de investigación crítica.
- ▶ **Generalización:** Un modelo óptimo descubierto para CIFAR-10 a menudo requiere ajustes para funcionar bien en imágenes médicas o satelitales.
- ▶ **Robustez:** Se ha demostrado que las arquitecturas generadas automáticamente son más susceptibles a ataques adversarios que las manuales.

Herramientas prácticas (Parte 1): AutoML y Prototipado

- ▶ **AutoKeras:** La puerta de entrada más amigable. Permite encontrar la mejor arquitectura para clasificación de imágenes, texto o datos tabulares con apenas tres líneas de código. Ideal para quienes buscan prototipado rápido sin ser expertos.
- ▶ **AutoGluon (AWS):** Una de las herramientas más potentes para llevar AutoML a producción. Destaca por crear automáticamente ensamblajes de redes neuronales de alta precisión, requiriendo muy poca configuración inicial por parte del usuario.

Fuentes y Referencias:

- Jin, H., et al. (2019). "Auto-Keras: An Efficient Neural Architecture Search System". *KDD*.
- Erickson, N., et al. (2020). "AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data". *arXiv*.

Herramientas prácticas (Parte 2): NAS Avanzado y Escala

- ▶ **Microsoft NNI (Neural Network Intelligence):** Un framework profesional que te da el control. Ya trae implementados algoritmos NAS modernos (como DARTS o ENAS) listos para integrarse directamente con tus modelos personalizados de *PyTorch* o *TensorFlow*.
- ▶ **Ray Tune:** La solución estándar cuando necesitas escalar. Si tienes acceso a múltiples GPUs o clusters en la nube, esta librería distribuye la búsqueda de arquitecturas e hiperparámetros de manera masiva y eficiente.

Fuentes y Referencias:

- Microsoft NNI: Documentación oficial (<https://github.com/microsoft/nni>).
- Liaw, R., et al. (2018). "Tune: A Research Platform for Distributed Model Selection and Training". *ICML AutoML Workshop*.

Herramientas prácticas (Parte 3): Optimización Multiobjetivo

- ▶ **Optuna:** Es la herramienta más popular hoy en día. Soporta optimización multiobjetivo de forma nativa. Por debajo utiliza algoritmos como **NSGA-II** para entregarte la Frontera de Pareto.
- ▶ **PyMOO / DEAP:** Si necesitas construir tu propio algoritmo evolutivo para una arquitectura de red muy exótica.

Fuentes y Referencias:

- Akiba, T., et al. (2019). "Optuna: A Next-generation Hyperparameter Optimization Framework". *KDD*.
- Blank, J. & Deb, K. (2020). "pymoo: Multi-Objective Optimization in Python". *IEEE Access*.

Conclusiones

- ▶ NAS representa la maduración de la Inteligencia Artificial: pasamos de ser *artesanos* a *arquitectos de algoritmos*.
- ▶ La optimización multiobjetivo nos garantiza que la inteligencia artificial se adapte al mundo real (hardware) y no solo gane competencias teóricas.
- ▶ **El futuro del ingeniero:** El valor ya no reside en diseñar la red neuronal a mano, sino en dominar la formulación matemática del espacio y el método de búsqueda.

¡Muchas gracias por su atención!

Contacto:

Dr. Cosijopii Garcia Garcia

cosijopii@unistmo.edu.mx

www.cosijopii.com